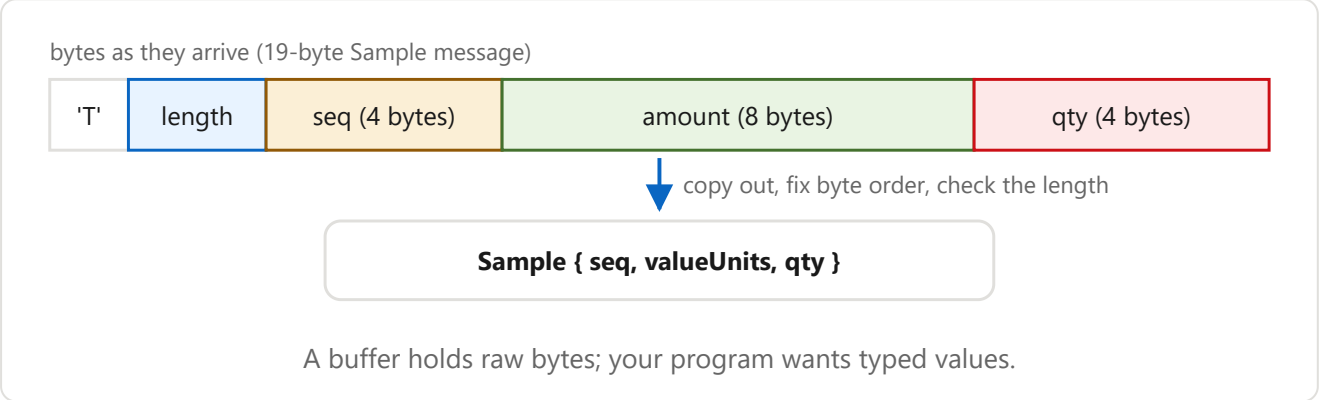
Binary Data and Parsing

How a program turns bytes from the network into values: bit operations, struct layout, byte order, safe reads, zero-copy parsing, amounts and sequence numbers. Compiled examples are in `examples/lessons/11-wire-formats`.

From bytes to values



A network message arrives as bytes. The job: turn them into values **quickly, correctly and without undefined behavior**.

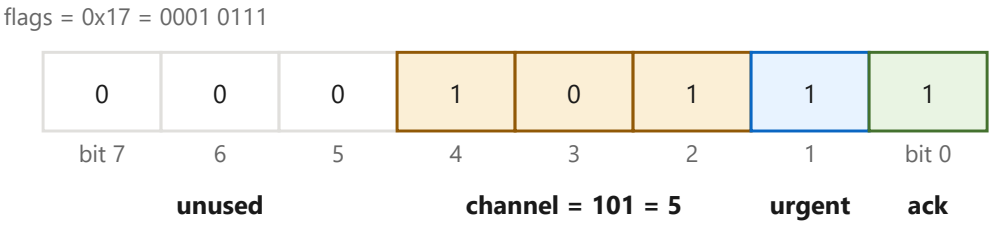
- **Never cast the buffer to a struct pointer.** Copy fields out instead.
- **Check the length before every read.** A truncated or corrupt packet must be rejected, not trusted.
- **Convert byte order explicitly.** The network and your CPU may disagree.

THE TEMPTING WRONG WAY

```
auto* t = reinterpret_cast<const Sample*>(buf);   // reads fields straight out of the buffer
std::cout << t->seq;
```

That one line hides four problems: **aliasing** (undefined behavior), **alignment** (the buffer address may not suit a `Sample`), **padding** (the struct layout may not match the wire) and **byte order**. The next sections take them one by one.

Integers and bit manipulation



Read the channel: (flags >> 2) & 0x7 shift the field down, then mask its width.

Protocols pack several fields into one byte; bit operations unpack them.

OPERATION	EXPRESSION
set bit n	`x | (1u << n)`
clear bit n	`x & ~(1u << n)`
toggle bit n	`x ^ (1u << n)`
test bit n	`(x >> n) & 1u`
power of two?	`x != 0 && (x & (x - 1)) == 0`
count set bits	`std::popcount(x)` (C++20), or loop `x &= x - 1` (clears the lowest set bit each time)

```cpp
constexpr std::uint8_t kAck = 1u << 0;
constexpr std::uint8_t kUrgent = 1u << 1;
constexpr unsigned channelOf(std::uint8_t f) { return (f >> 2) & 0x7u; }

std::uint8_t flags = kAck | kUrgent | (5u << 2);      // 0x17
bool ack = flags & kAck;                             // true
unsigned channel = channelOf(flags);                  // 5
```

Rules that bite

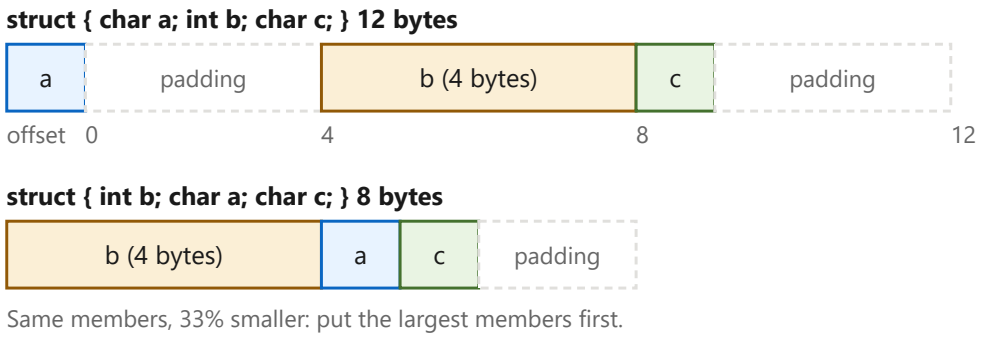
- Use **fixed-width types** (`std::uint8_t`, `std::uint32_t`, `std::int64_t`) for anything on the wire; `int` and `long` change size between platforms (`long` is 32-bit on Windows).
- **Unsigned overflow wraps** (well defined); **signed overflow is undefined behavior**.
- Shifting by the type's width or more is undefined: write `1u << n` with `n < 32`. Do bit work on **unsigned** types.
- Small types are promoted to `int` in arithmetic. A plain `char` may be signed, so widening `char(0xFF)` to `int` gives **-1**: read bytes as `unsigned char` or `std::uint8_t`.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 4.8 The bitwise operators (PDF pages 181 to 184)
- 17.2 The bitset type (PDF pages 752 to 756)

Struct layout, alignment and padding



Same members, 33% smaller: put the largest members first.

The compiler inserts padding so every member sits at a multiple of its alignment.

Each type has an **alignment** (`alignof`): an `int` must start at a multiple of 4, a `double` or `int64_t` at a multiple of 8. The compiler pads between members to guarantee that, and pads the end so that arrays of the struct stay aligned. So `sizeof` is often bigger than the sum of the members.

```
struct Careless { char a; int b; char c; };          // 12 bytes
struct Ordered  { int b; char a; char c; };          //  8 bytes

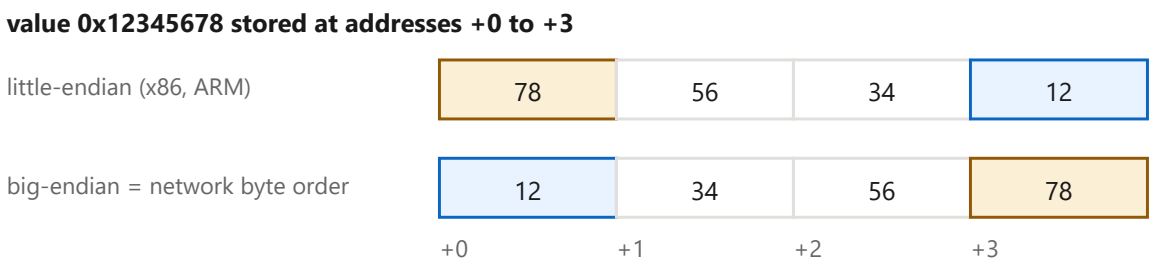
static_assert(sizeof(Careless) == 12 && alignof(Careless) == 4);
static_assert(offsetof(Careless, b) == 4);           // offsetof gives the
byte offset of a member
```

Packing, and why not to rely on it

```
#pragma pack(push, 1)
struct Packed { char a; int b; char c; };            // 6 bytes, no padding:
matches a wire layout
#pragma pack(pop)
```

> PACKING IS NOT FREE A packed member may be **unaligned**: slower on some CPUs, a fault on others, and taking a reference or pointer to it is unsafe. Do not read the wire into a packed struct and use it directly. Read the bytes with `memcpy` (next sections) and keep ordinary structs inside your program. `static_assert(sizeof( ... ))` is a cheap guard against accidental layout changes.

Endianness



Little-endian stores the least significant byte first, big-endian the most significant first.

Same value, different byte order in memory.

- **Endianness** is the order of a multi-byte value's bytes in memory. Network protocols traditionally use **big-endian** ("network byte order"); most desktop and server CPUs are little-endian. Protocols differ, so read each protocol's specification.
- **Convert at the boundary**: when bytes enter the program, turn them into native values, and stay native inside.
- Tools: `ntohl` / `htonl` (sockets), `std::endian::native` (C++20) to test the CPU, `std::byteswap` (C++23). The portable way that works everywhere is to **assemble the value from bytes with shifts**:

```
constexpr std::uint32_t readBe32(const unsigned char* p) {
    return (std::uint32_t{p[0]} << 24) | (std::uint32_t{p[1]} << 16)
         | (std::uint32_t{p[2]} <<  8) |  std::uint32_t{p[3]};
}
const unsigned char wire[] = {0x12, 0x34, 0x56, 0x78};
readBe32(wire);                                    // 0x12345678 on ANY CPU
```

Widen each byte to `uint32_t` *before* shifting, otherwise it promotes to a signed `int` and shifting a large value into the top bit is a trap. The same code works on little- and big-endian machines, and compilers turn it into a single load plus a byte swap.

Reading bytes safely: aliasing and memcpy

Strict aliasing: you may access an object only through a pointer or reference of its own type (or a few related ones). Reading a `uint32_t` out of memory that holds something else is undefined behavior, and the optimizer is allowed to assume it never happens.

TECHNIQUE	DEFINED?	NOTES
`*reinterpret_cast<const uint32_t*>(buf + 1)`	**no**	aliasing violation, and `buf + 1` is probably misaligned
`std::memcpy(&v, buf + 1, sizeof v)`	yes	compilers turn it into a plain load; no alignment requirement on `buf`
`std::bit_cast<To>(from)` (C++20)	yes	same size, trivially copyable types, e.g. float to its bits
assemble with shifts (`readBe32`)	yes	also fixes byte order
write through a `union` and read another member	**no** in C++	works as a GCC extension, but is not portable C++
view an object as `unsigned char`, `char` or `std::byte`	yes	the one exception: these may alias anything (but not the reverse)

```
std::uint32_t v;
std::memcpy(&v, buf + 1, sizeof v);                 // native byte order: add a swap if the wire is big-endian

std::bit_cast<std::uint32_t>(1.0f);                 // 0x3f800000: the bits of a float
```

KEY POINT "Why is casting the receive buffer to a struct pointer wrong?" Aliasing (UB), alignment, padding that may not match the wire, and byte order. "What do you do instead?" `memcpy` or shifts per field, after checking the length.

Zero-copy parsing

A **zero-copy** parser never copies the message: it keeps views (`std::string_view`, `std::span`) into the receive buffer and a cursor that advances. No allocation, no copying, no exceptions on the hot path.

```
struct Field { int tag; std::string_view value; };

// "1=D|2=north|3=12345|"  →  (1,"D")  (2,"north")  (3,"12345")
std::optional<Field> nextField(std::string_view& rest) {
    auto bar = rest.find('|');
    if (bar == std::string_view::npos) return std::nullopt;       //
incomplete: reject
    std::string_view item = rest.substr(0, bar);
    rest.remove_prefix(bar + 1);                                   // advance
the cursor
    auto eq = item.find('=');
    if (eq == std::string_view::npos) return std::nullopt;
    Field f{};
    auto r = std::from_chars(item.data(), item.data() + eq, f.tag);
    if (r.ec ≠ std::errc{}) return std::nullopt;
    f.value = item.substr(eq + 1);                                 // a view:
no copy
    return f;
}
```

- `std::from_chars` (C++17) converts text to numbers with **no allocation, no exceptions and no locale**; it returns an error code and the pointer where it stopped. Compare with `stoi` (throws, needs a `std::string`) and `stringstream` (slow).
- **Validate everything** before trusting it: length fields, tags, digits. Corrupt input is normal; return `std::nullopt` or an error code instead of throwing.
- A view is valid only while the **buffer is alive and unchanged**. If the buffer is reused for the next packet, copy out what you need first.

Money amounts: fixed-point versus double

```
0.1 + 0.2 == 0.3;       // false
0.1 + 0.2;              // 0.30000000000000004
```

`double` cannot represent most decimal fractions exactly, so sums and comparisons drift. For money and amounts use an **integer count of units**: 101.2500 with a unit of 0.0001 is `1012500`.

```
class Money {
    std::int64_t units_;                                    // 1 unit = 0.0001
public:
    static constexpr std::int64_t kScale = 10000;
    constexpr Money operator+(Money o) const { return Money(units_ +
o.units_); }
    constexpr auto operator<=>(const Money&) const = default;    // exact
comparison, no epsilon
    static std::optional<Money> parse(std::string_view s);      // "101.25" →
1012500, rejects a 5th decimal
};
// 0.1 + 0.2 == 0.3 is true here: 1000 + 2000 == 3000
```

- **Exact and deterministic**: the same inputs give the same bits on every machine, and equality needs no epsilon.
- **Parse from text directly** into units; never go through `double`.
- **Watch overflow**: `int64_t` holds about 9.2e18 units (9.2e14 amount units at 4 decimals). Multiplying amount by quantity can exceed that: check, or use a wider type (`__int128` in GCC and Clang).
- Decide the rounding rule (down, half-up, half-even) once, and apply it in one place.

Sequence numbers and gaps



Each message carries a sequence number; a jump means something was lost.

RECEIVED VERSUS EXPECTED	MEANING	TYPICAL ACTION
equal	in order	process, expect `seq + 1`
higher	**gap**: messages were lost	buffer new messages, request a retransmission or a snapshot
lower	duplicate or old	ignore

```
enum class SeqResult { InOrder, Duplicate, Gap };

SeqResult classify(std::uint32_t expected, std::uint32_t received) {
    auto diff = static_cast<std::int32_t>(received - expected);   // unsigned subtraction wraps, then read it as signed
    if (diff == 0) return SeqResult::InOrder;
    return diff < 0 ? SeqResult::Duplicate : SeqResult::Gap;
}
```

Wrap-around: a 32-bit counter returns to 0 after 4294967295. Comparing with `<` breaks at that point; the unsigned subtraction followed by a signed reinterpretation keeps working as long as the two values are within 2^{31} of each other. Expecting 4294967295 and receiving 0 is a **gap of one**, not a duplicate.